

Zewelian Life Project Report

This report provides an overview of the features implemented in the Zewelian Life project using C++ and the SFML (Simple and Fast Multimedia Library). The project aims to create an immersive 2D simulation game where players can experience a virtual world, interact with characters, explore environments, and complete various objectives.

Features:

Main Menu:

the main menu feature, which involves variables `int selected` stores the currently selected option in the main menu, `sf::Font font`, `sf::Event event` is used to capture user input or events and It allows for user interaction, such as selecting menu items or entering text, `sf::Text mainMenu[]` array holds the text elements of the main menu and It contains the options available in the main menu, such as "Start Game," "Controls," or "Quit.", `int page`, `std::string name` stores the player's name input and It holds the text entered by the player when prompted for their name, and member functions `menu()` sets up the main menu with specified width and height and It initializes the menu options, positions them on the screen, and prepares the layout, `draw()`, `moveUp()`, `moveDown()`, `setSelected()` function sets the selected menu option to a specific index `n` and It allows for programmatically changing the selected option, `isPressed()` checks if a specific key or event has been triggered by the user, and `playerName()` prompts the player to enter their name. It displays a text input field where the player can type their name using the provided window

Map:

the map feature, which involves variables `sf::Sprite b1` represents the visual background of the map and It holds the image data and properties required to display the map's background on the screen., `sf::Texture t1` represents the graphical appearance of the map's background and It holds the image data that will be displayed as the map's visual representation

, and member functions `initTexture()` is responsible for initializing the texture used for the map's background, `initBackground()`, `render()`, and `updateBackgroundBoundsCollision()` checks for collisions or interactions between the player and the map's background and It ensures that the player remains within the boundaries of the map and prevents them from moving outside the playable area

Player:

the player feature, which involves variables `bool enter`, `int timer`, `sf::Clock clock` used to track elapsed time or control the timing of certain events, `sf::Sprite sprite`, `sf::Texture texture`, `int x` and `int y` are hold the player's current position on the game screen, `float movementSpeed` determines the speed at which the player moves and It controls the rate at which the player's position is updated during movement, and member functions `enter()` and `updateInput()` handle player input and events, such as keyboard or mouse input. They capture and interpret input from the player to control the actions or movements of the character, `updateWindowBoundsCollision()` checks for collisions between the player and the boundaries of the game window and It ensures that the player remains within the visible game area and prevents them from going outside the screen. `update()`, `render()`, `setPosition()` sets the player's position to the specified coordinates (x, y) on the game screen, `initTexture()`, `initSprite()`, and `initVariables()`.

Cutscene:

the cutscene feature, which involves variables `sf::Sprite (bus)`, `sf::Texture (texture)`, `int (x, value)`, and `bool (enter)`, as well as member functions `initTexture()` is responsible for initializing the texture used for the cutscene's visual element, `initSprite()` is responsible for initializing the sprite object associated with the cutscene., `initVariables()`, `render()`, `busScene()` and `enter()` and `updateInput()` are used to handle specific cutscene actions or player interactions.

Sound:

The sound feature, which involves a variable `sf::Music` and member functions `setLoop()` is used to set whether the music should loop or not.

, playMusic() is responsible for playing the loaded music and It starts the playback of the music stored in the music variable.

, and stopMusic() is used to stop the playback of the music and It halts the music playback and sets the playback position to the beginning.

NPC:

The NPC feature involves rendering NPCs on the game window, allowing them to interact with the player and contribute to the immersive gameplay experience.

Items:

Items feature involves the usage of variables sf::Texture and sf::Sprite to represent the item's visual appearance, and member functions initTexture(), initSprite() that is responsible for initializing the texture used for the item's visual appearance, setPosition() is used to set the position of the item within the game world and It takes the X and Y coordinates as arguments to define the item's position, pickItem() a member function represents the interaction or action of picking up the item., and render() to initialize, position, interact with, and render the items.

Buttons:

Button feature involves the usage of variables sf::Font and sf::Text to represent the button's text content, and member functions setFont() which setting the font used for the button's text.

, setText() which is responsible for setting the content and properties of the button's text., and render() to initialize and render the button.

Animation:

Animation feature, which involves a texture variable and functions for initializing the texture, sprite, and variables that include variables such as animation frame duration, position and size update Animation within this

function, the animation frame is changed, typically by modifying the texture or sprite properties